- You can override the default binding source for a parameter by applying `[From*]` attributes to your handler parameters, such as `[FromHeader]`, `[FromQuery]`, `[FromBody]`, and `[FromServices]`. These parameters take precedence over convention-based assumptions.
- You can encapsulate an endpoint handler's parameters by creating a type containing all the parameters as properties or a constructor argument and decorate the parameter with the `[AsParameters]` attribute. This approach can help you simplify your endpoint's method signature.
- Validation is necessary to check for security threats. Check that data is formatted correctly, confirm that it conforms to expected values and verify that it meets your business rules.
- Minimal APIs don't have built-in validation APIs, so you typically apply validation via a minimal API filter. This approach provides flexibility, as you can implement validation in the way that suits you best, though it typically means that you need to use a third-party package.
- The MinimalApis.Extensions NuGet package provides a validation filter that uses `DataAnnotations` attributes to declaratively define the expected values. You can add the filter with the extension method `WithParameterValidation()`.
- To add custom validation of simple types with MinimalApis.Extensions, you must create a containing type and use the `[AsParameters]` attribute.

---

[1] I have a series discussing strongly-typed IDs and their benefits on my blog at http://mng.bz/a1Kz.